CS5013 Project Proposal

GUI Framework for Multi-Tool Panoramic Image Segmentation

Rathan Balaji P R
Roll No. CS26E009
CSE, IIT Madras

August 21, 2026

1 Team information

Team ID: solo

- **Member 1:** Rathan Balaji P R – Roll no. CS26E009 – Department of Computer Science and Engineering – GitHub handle: Rathan Balaji

Primary contact email: cs26e009@smail.iitm.ac.in

2 Problem statement

Panoramic image segmentation is used to assign semantic or instance-level labels to regions of a 360° image, but available segmentation models differ substantially in their input requirements, inference interfaces, output formats, and supported segmentation tasks. For a user working with panoramic images, running several segmentation tools and comparing their outputs therefore requires separate model-specific workflows and manual handling of results. The proposed project addresses this problem by providing a single interface through which multiple heterogeneous segmentation tools can be executed on the same panoramic input and their outputs can be visualized and compared in a standardized manner. The system will also provide a modular integration mechanism so that additional segmentation tools can be added without redesigning the complete application.

3 Users and stakeholder

Named stakeholder: Dr. Sukhendu Das, Professor, Department of Computer Science and Engineering, IIT Madras.

The project was discussed with Dr. Sukhendu Das in the context of developing a practical software tool for image segmentation research and experimentation. He proposed the project direction of developing a GUI for multi-tool segmentation and provided relevant material describing the expected functionality and research context. The proposed system is intended to support researchers and students who need to run different segmentation tools on panoramic images and inspect or compare their outputs without maintaining separate user workflows for every model. Dr. Sukhendu Das has agreed to act as the stakeholder for the project. The stakeholder acknowledgement email will be attached as Appendix A.

Date of first project discussion: 9th August 2026.

4 Prior work and why it does not suffice

Several existing platforms provide parts of the functionality required by the proposed system, including interactive segmentation, extensible plugins, and visualization. However, they do not provide the

complete combination of heterogeneous segmentation-tool integration, panorama-aware processing, and unified comparison targeted by this project.

1. **MATLAB Image Segmenter** – MathWorks Image Segmenter. **Overlap:** Provides an interactive GUI for image segmentation using multiple segmentation approaches and supports visualization of segmentation results. **Gap:** It is primarily part of the MATLAB image-processing ecosystem and is not designed as a lightweight framework for integrating independently implemented panoramic segmentation models with different runtimes, preprocessing pipelines, and output formats.
2. **napari** – napari. **Overlap:** Provides an interactive image viewer with a plugin architecture that can be extended with image-analysis and segmentation tools. **Gap:** Although napari provides general-purpose plugin support, the proposed project focuses specifically on a standardized adapter layer for heterogeneous panoramic segmentation models, including model-specific preprocessing, execution, output normalization, panorama-aware visualization, and cross-model quantitative comparison.
3. **3D Slicer** – 3D Slicer. **Overlap:** Provides a graphical environment for image visualization and segmentation and supports extensible AI-based segmentation workflows. **Gap:** Its primary focus is medical and volumetric image computing rather than experimentation with multiple independent 360° panoramic segmentation models. It does not provide the proposed panorama-specific workflow for standardized execution and comparison of heterogeneous segmentation outputs.

The proposed project therefore focuses on the integration layer between the user and the segmentation tools. Its distinguishing functionality is the combination of a common model-adapter interface, panorama-aware processing and visualization, standardized segmentation outputs, and unified comparison of multiple segmentation tools.

5 Tech stack

Language: Java, as required by the course policy.

- **Build system:** Maven.
- **GUI framework:** JavaFX for the desktop graphical interface.
- **Core Java libraries:** Java HTTP Client for service communication, Jackson for structured configuration and result serialization, Java NIO for file and path management, and Java ProcessBuilder for controlled execution of external model processes.
- **Integration mechanism:** A Java-based adapter/plugin layer will provide a common interface to individual segmentation tools. Where a model is implemented in a different runtime, the Java application will invoke or communicate with the corresponding inference service through a controlled process or HTTP interface.
- **Visualization:** JavaFX image/canvas components for displaying the panoramic input, segmentation masks, overlays, and comparison results.
- **Execution environment:** Initially the stakeholder's or project workstation with GPU-enabled model runtimes where required. The architecture will keep the GUI and model execution layer separated so that model execution can later be moved to a department server if required.

A central design principle is the use of a common abstraction such as a `SegmentationTool` interface. Each integrated model will be represented by an adapter responsible for its model-specific preprocessing, execution, post-processing, and conversion of results into the common representation consumed by the GUI.

6 AI-usage statement

We will use AI coding assistants such as ChatGPT, GitHub Copilot, Claude Code, or Gemini during this project, in accordance with the course policy on AI-assisted programming. AI assistance may be used for code scaffolding, understanding APIs, debugging, testing, documentation, and exploring implementation approaches.

All AI-generated code will be reviewed, tested, and adapted before being included in the project. The project member will understand the submitted code and be able to explain the implementation and design decisions during the viva. AI tools will be used as development assistance and not as a substitute for understanding or verification.

7 Verification plan

Test fixtures we will use:

- A representative set of equirectangular 360° indoor panoramic images from the project dataset.
- Panoramic images containing different scene layouts, object scales, boundaries, and classes so that the framework is tested beyond a single visual configuration.
- Ground-truth segmentation masks, where available, for quantitative evaluation of the integrated tools and validation of the computed comparison metrics.
- Synthetic and deliberately difficult cases covering invalid image paths, unsupported image formats, missing model configuration, model execution failure, empty segmentation output, and mismatched output dimensions.
- At least one test case for each integrated segmentation tool to verify the adapter contract and standardized output conversion.

User-visible acceptance criteria:

- A user can load a supported 360° panoramic image, select an integrated segmentation tool, execute it, and view the resulting segmentation overlay through the GUI without manually invoking the model's command-line workflow.
- A user can select multiple integrated tools for the same panoramic image and view their outputs in a common comparison interface using a consistent visualization format.
- The framework converts outputs from the integrated tools into a standardized representation while preserving the relevant class/instance information required for comparison.
- Where ground truth is available, the system reports quantitative comparison measures such as mean IoU, pixel accuracy, Dice/F1 score, and per-class IoU, together with the corresponding visual results.
- Adding a new segmentation tool requires implementing the defined adapter/interface and configuration rather than modifying the core comparison and GUI workflow.
- The application reports model or input failures clearly instead of silently producing an invalid segmentation result.

Verification approach. Unit tests will validate configuration handling, adapter contracts, file validation, result parsing, and metric calculations. Integration tests will execute each model adapter on fixed test images and verify that the standardized result is produced. End-to-end tests will exercise the complete workflow from panoramic image selection through inference, visualization, and comparison. Quantitative results will be checked against independently computed reference metrics for selected test cases.

8 Milestone plan

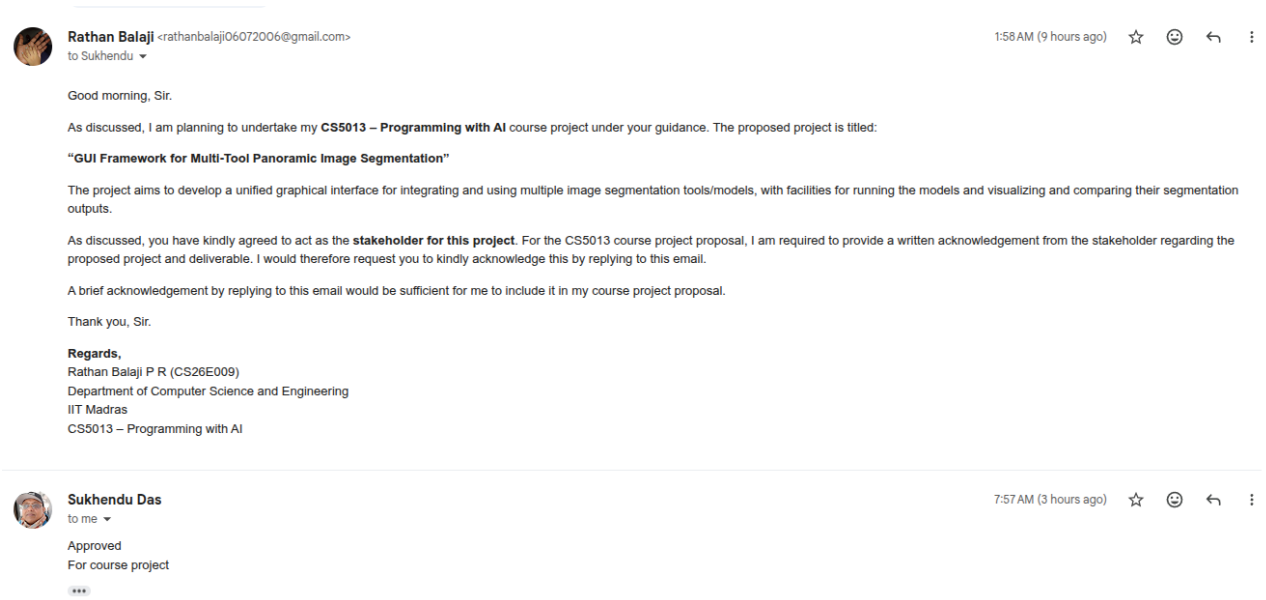
- **By the design doc (11 Sep 2026):** Finalize the requirements and architecture; define the common segmentation-tool interface, panoramic data representation, comparison format, and GUI workflow; implement a prototype GUI and one end-to-end model adapter.
- **By the mid-demo (9 Oct 2026):** Demonstrate an end-to-end workflow on real panoramic images with multiple integrated segmentation tools, including standardized result visualization and an initial comparison module.
- **By the final submission (6 Nov 2026):** Deliver the complete runnable GUI with the selected segmentation-tool adapters, panorama visualization, quantitative comparison where ground truth is available, automated tests, documentation, and instructions for integrating an additional tool.

9 Risks and plan B

1. **Risk:** Some segmentation models may have incompatible dependencies, GPU requirements, or Python-based runtimes that cannot be embedded directly in the Java application. **Plan B:** Keep model execution outside the Java GUI and communicate through a defined process or HTTP adapter boundary. If a model cannot be integrated reliably, reduce the initial model set while preserving the common plugin architecture.
2. **Risk:** Different models may produce incompatible segmentation representations, class identifiers, or image dimensions. **Plan B:** Define a canonical segmentation-result schema and implement model-specific post-processing adapters that map outputs into that schema. Unsupported information will be explicitly marked rather than silently discarded.
3. **Risk:** Panoramic distortion may make direct visual comparison misleading, particularly near the horizontal wrap-around boundary. **Plan B:** Include panorama-aware visualization and, where required, evaluate selected regions or transformed views in addition to the complete equirectangular representation.
4. **Risk:** Model execution time may be too high for interactive use. **Plan B:** Run inference asynchronously, provide progress/error states, cache completed results, and prioritize a small set of representative models for the live demonstration.
5. **Risk:** Ground-truth masks may not be available for every panoramic image or may use label definitions that are not directly compatible with every integrated segmentation tool. **Plan B:** Separate visual comparison from quantitative evaluation and compute metrics only for samples with valid ground truth and compatible label mappings.

A Stakeholder acknowledgement email

The following screenshot shows the stakeholder's acknowledgement regarding the proposed project.



Rathan Balaji <rathanbalaji06072006@gmail.com>
to Sukhendu

1:58 AM (9 hours ago)

Good morning, Sir.

As discussed, I am planning to undertake my **CS5013 – Programming with AI** course project under your guidance. The proposed project is titled:

"GUI Framework for Multi-Tool Panoramic Image Segmentation"

The project aims to develop a unified graphical interface for integrating and using multiple image segmentation tools/models, with facilities for running the models and visualizing and comparing their segmentation outputs.

As discussed, you have kindly agreed to act as the **stakeholder for this project**. For the CS5013 course project proposal, I am required to provide a written acknowledgement from the stakeholder regarding the proposed project and deliverable. I would therefore request you to kindly acknowledge this by replying to this email.

A brief acknowledgement by replying to this email would be sufficient for me to include it in my course project proposal.

Thank you, Sir.

Regards,
Rathan Balaji P R (CS26E009)
Department of Computer Science and Engineering
IIT Madras
CS5013 – Programming with AI

Sukhendu Das
to me

7:57 AM (3 hours ago)

Approved
For course project

Figure 1: Stakeholder acknowledgement from Dr. Sukhendu Das.